# Regression Modeling and Rule Discovery using Machine Learning

**Rutvikkumar Babubhai Balar, r.balar@oth-aw.de**[1]

[1]*OTH Amberg-Weiden, Study programme "Artificial Intelligence for Industrial Applications"*

Report written on January 12, 2026

**Abstract**

This report addresses two complementary machine learning tasks based on the same dataset.
**In the first part,** a supervised regression model is developed to predict a continuous target variable (`target01`). Due to strong non-linear relationships in the data, a CatBoost regressor combined with degree–3 polynomial feature engineering is employed. Feature selection and polynomial expansion are performed within a nested cross-validation framework to prevent data leakage. The final model achieves an average coefficient of $R^2 = 0.9533$ with a small train–validation gap, indicating strong predictive performance and good generalization capability.
**In the second part,** the focus shifts from prediction to rule discovery. Instead of relying on a black-box model, explicit if–then–else rules are derived to deterministically reproduce a second target variable (`target02`). Through systematic feature analysis, threshold identification, and validation, a compact set of four rules is obtained. These rules depend on a single switcher feature and simple linear computations, and they reproduce the target exactly ($R^2 = 1.0$), up to numerical precision.

## 1. Predicting target01 (Regression)

### 1.1. Introduction

The objective of this work is to predict a continuous target variable, `target01`, based on a set of numerical input features. The problem is formulated as a supervised regression task. The provided dataset consists of 10,000 labeled training samples and a separate evaluation dataset of equal size. Each sample is represented by 273 numerical features.

#### *1.1.1. Task Definition*

- **Learning Type:** Supervised regression
- **Input:** 273 numerical features per sample
- **Output:** Predicted values of `target01`

#### *1.1.2. Dataset Description*

The dataset is provided in the form of three CSV files. The training feature dataset.csv consists of 10,000 samples, each represented by 273 numerical input features. The corresponding target values for the training data are supplied in a separate labels file(target.csv).

In addition, an evaluation dataset of equal size (10,000 samples) is provided. This dataset contains the same 273 input features but does not include target labels. It is used exclusively for generating final predictions after model selection and validation.

### 1.2. Data Inspection

#### *1.2.1. Data Quality Checks*

All 273 input features are continuous numerical variables with consistent data types. No missing values or duplicate samples were detected in the training or evaluation datasets. As a result, no imputation, encoding, or data type conversion was required.

#### *1.2.2. Target Variable Analysis*

The target variable `target01` is continuous and bounded within a finite range. Basic descriptive statistics indicate a mean of 0.387 and a standard deviation of 0.236 and Target01 Range is [-0.04, 1.08]. No extreme outliers or anomalies were observed that would require special preprocessing.

#### *1.2.3. Sanity Check for Information Leakage*

A correlation analysis between each input feature and the target variable was performed to detect potential information leakage. The maximum observed absolute correlation was 0.157, indicating that no single feature directly encodes the target variable. Therefore, accurate prediction requires learning relationships among multiple features rather than relying on trivial shortcuts.

### 1.3. Methodology

#### *1.3.1. Overview of the Approach*

The solution follows a three-stage pipeline: feature selection, polynomial feature engineering, and gradient boosting regression. All transformations are performed inside cross-validation folds to prevent data leakage.

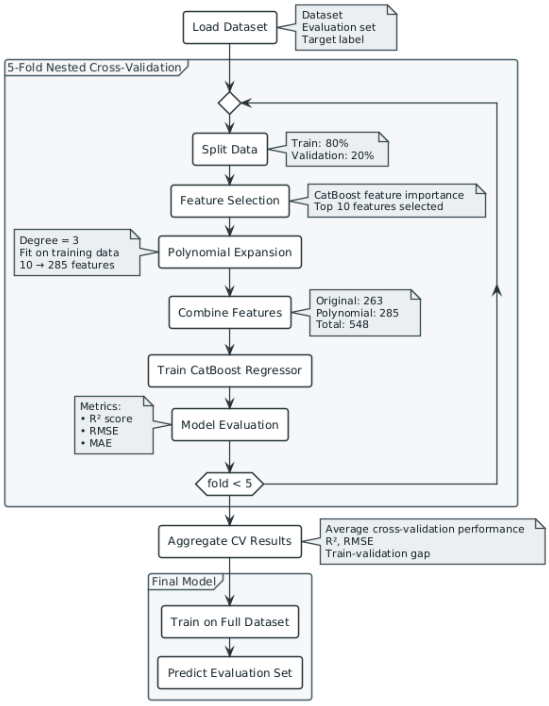



**Figure 1.** Pipeline Overview

**Pipeline summary:**

1. Select top 10 features using CatBoost importance scores.
2. Generate degree-3 polynomial features from selected features.
3. Combine with remaining original features.
4. Train CatBoost regressor on the combined feature set.

#### *1.3.2. Model Selection*

The CatBoost Regressor was selected based on a systematic baseline comparison:

The poor performance of linear models ($R^2 \approx 0.05$) combined with weak individual feature correlations (max 0.157) confirmed that the target depends on non-linear feature interactions.

**Table 1.** Baseline Model Comparison

| Model Family | Best $R^2$ | Observation |
|---|---|---|
| Linear (Ridge, Lasso) | 0.05 | Relationships are not linear |
| Kernel (SVR) | 0.08 | Ineffective in 273 dimensional space |
| Distance-based (KNN) | $< 0.05$ | Curse of dimensionality |
| Tree Ensemble (RF) | 0.09 | Limited interaction modeling |
| Gradient Boosting | **0.77** | Best baseline performance |

Among gradient boosting implementations, CatBoost outperformed XGBoost and LightGBM on this dataset.

**Final model configuration:**

**Table 2.** Final Hyperparameters

| Parameter | Value | Rationale |
|---|---|---|
| `n_estimators` | 800 | Optimal value identified via grid search |
| `learning_rate` | 0.05 | Trade-off between convergence speed and accuracy |
| `depth` | 10 | `depth=12` showed signs of overfitting |

#### 1.3.3. Feature Engineering

Polynomial features (degree 3) were introduced to capture non-linear interactions that the baseline CatBoost could not learn directly.

- **Why polynomial features:** Baseline CatBoost achieved $R^2 = 0.77$, suggesting missing feature interactions. Adding degree-3 polynomials increased $R^2$ to 0.9533.

**Why degree 3:**

**Table 3.** Polynomial Degree Analysis

| Degree | CV $R^2$ | Train-Val Gap |
|---|---|---|
| 2 | 0.9282 | 0.070 |
| **3** | **0.9533** | **0.045** |

Degree 3 achieved higher accuracy with a *smaller* generalization gap, indicating it captures the true underlying pattern rather than overfitting.

**Why top 10 features only:**

- Expanding all 273 features would create $>$ 3 million polynomial terms.
- Top 10 features capture 85% of cumulative importance.
- Reduces noise while retaining predictive power.

**Table 4.** Feature Importance

| Rank | Feature | Importance |
|---|---|---|
| 1 | feat_97 | 28.1% |
| 2 | feat_137 | 25.9% |
| 3 | feat_162 | 15.8% |
| 4–10 | feat_184, 60, 84, 66, 25, 240, 59 | 14.4% |

**Final feature set:** 548 features (263 original + 285 polynomial terms).

#### 1.3.4. Validation Strategy

5-fold nested cross-validation was used to obtain unbiased performance estimates.
**Why nested CV:** Feature selection and polynomial fitting depend on data. Performing these steps on the full dataset before CV would leak information from validation folds into the transformation, producing optimistic estimates.

**Leakage-free procedure:**

For each fold $k = 1, \ldots, 5$:

1. Feature selection → fit on training fold only
2. Polynomial fitting → fit on training fold only
3. Model training → fit on transformed training data
4. Evaluation → predict on transformed validation data

This ensures validation predictions are made on data the model has never seen during any stage of the pipeline.

**Computational cost:** Approximately 30 minutes for full validation. This cost is accepted for reliable, leakage-free estimates.

### 1.4. Results

#### 1.4.1. Cross-Validation Performance

The final model demonstrates strong predictive performance under 5-fold nested cross-validation. The mean coefficient of determination ($R^2$) across folds is 0.9533, indicating that approximately 95.3% of the variance in the target variable is explained by the model.

The corresponding root mean squared error (RMSE) of 0.0510 and mean absolute error (MAE) of 0.0384 indicate low average prediction errors relative to the target range. These results confirm that the model accurately captures the relationship between the input features and the target variable.

#### 1.4.2. Stability and Generalization

To evaluate generalization behavior, model performance was analyzed across individual cross-validation folds. Figure 2 summarizes fold-wise training and validation metrics, including $R^2$ scores, RMSE values, and the train–validation performance gap.

| Fold | Train R² | Val R² | Gap | Train RMSE | Val RMSE |
|---|---|---|---|---|---|
| 1 | 0.9992 | 0.9597 | 0.0395 | 0.0065 | 0.0472 |
| 2 | 0.9992 | 0.9485 | 0.0507 | 0.0064 | 0.0542 |
| 3 | 0.9993 | 0.9550 | 0.0443 | 0.0065 | 0.0496 |
| 4 | 0.9993 | 0.9508 | 0.0485 | 0.0063 | 0.0523 |
| 5 | 0.9993 | 0.9528 | 0.0465 | 0.0061 | 0.0514 |
| **Mean** | **0.9993** | **0.9533** | **0.0459** | **0.0064** | **0.0510** |

**Figure 2.** 5-fold nested cross-validation

Validation $R^2$ values remain consistently high across all folds, with all values exceeding 0.94 and a mean validation $R^2$ of 0.9533. The train–validation gap remains small and stable, with a mean gap of approximately 0.045, indicating limited overfitting.

Overall, the low standard deviation across validation folds (0.0038) and the consistent train–validation gap indicate stable generalization and low sensitivity to data partitioning, supporting reliable generalization to unseen data.

#### 1.4.3. Prediction Quality

Figure 3 shows the relationship between the actual target values (`target01`) and the corresponding cross-validated predictions produced by the final model. Each point represents one validation sample, while the red dashed line indicates the ideal case where predicted values exactly match the true values. The predicted values closely follow the diagonal reference line over the full range of the target variable. This indicates that the model achieves accurate predictions not

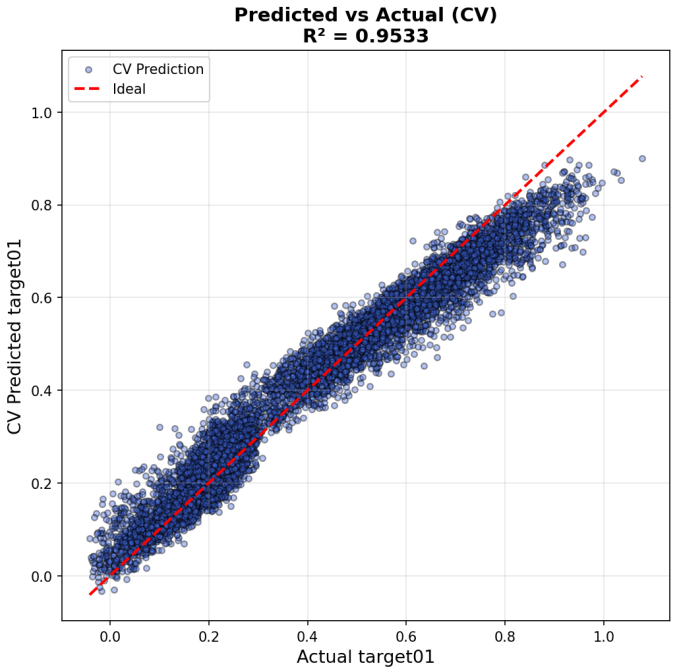



**Figure 3.** Predicted vs. Actual Target Values

only on average, but also consistently for both lower and higher target values. No systematic deviation or nonlinear distortion is visible, suggesting that the model has successfully captured the underlying relationship between the input features and the target.

Furthermore, the tight clustering of points around the diagonal confirms the low prediction error observed in the quantitative evaluation. The achieved coefficient of determination ($R^2 = 0.9533$) shows that the model explains the majority of the variance in `target01`. Overall, this visual comparison confirms that the model generalizes well within the nested cross-validation framework and produces stable, high-quality predictions.

### 1.5. Limitations

Although the proposed approach shows strong predictive performance, several limitations should be considered.

#### *1.5.1. Model interpretability*

The final model is based on a large ensemble of boosted decision trees combined with polynomial feature expansion. This results in a complex model structure, making it difficult to interpret individual predictions. The influence of specific input features on the output is therefore not easily explainable. In Part 1 of the task, the main focus was placed on achieving high predictive accuracy rather than model interpretability.

#### *1.5.2. Computational cost*

The nested cross-validation pipeline requires relatively high computational effort and takes approximately 30 minutes to complete. While this runtime is acceptable for offline evaluation and model selection, it limits rapid experimentation. Additionally, this approach would be less suitable for applications that require frequent retraining or real-time predictions.

#### *1.5.3. Feature engineering scope*

In this work, only polynomial feature interactions were explored. Other feature transformation techniques, such as logarithmic or trigonometric transformations, were not investigated. Including such transformations could potentially lead to further performance improvements.

#### *1.5.4. Generalization assumptions*

The reported results assume that the evaluation data follows a similar distribution to the training data. If a significant distribution shift occurs, the predictive performance of the model may decrease. This potential issue was not explicitly addressed within the scope of the current study.

## 2. Part 2: Rules discovery

### 2.1. Introduction

The goal of Part 2 is to discover deterministic *if–then–else* rules that exactly predict the `target02` variable. According to the task specification, `target02` depends on only a small number of input features, and a set of simple rules exists that can be used to compute it accurately.

In contrast to Part 1, predictions must be implemented using explicit conditional logic rather than trained machine learning models. This constraint reflects a realistic deployment scenario on an edge device, where conventional machine learning inference is not allowed and only basic operations are permitted.

For this reason, the task was approached as a rule discovery problem rather than a predictive modeling task. The objective was not to obtain an approximate solution, but to reconstruct the underlying data-generating rules exactly. Success is therefore measured by identifying the correct features, the correct threshold conditions, and the correct calculation formulas, as well as by validating that these rules reproduce the target values consistently.

### 2.2. Feature Identification

To identify the relevant features among the 273 available inputs, eight independent feature selection methods were applied. These included tree-based importance measures (Decision Tree (DT), Random Forest (RF), Extra Trees (GB), Gradient Boosting (GB), and AdaBoost (Ada)) as well as statistical approaches (Mutual Information (MI), F-Regression, and Pearson correlation).

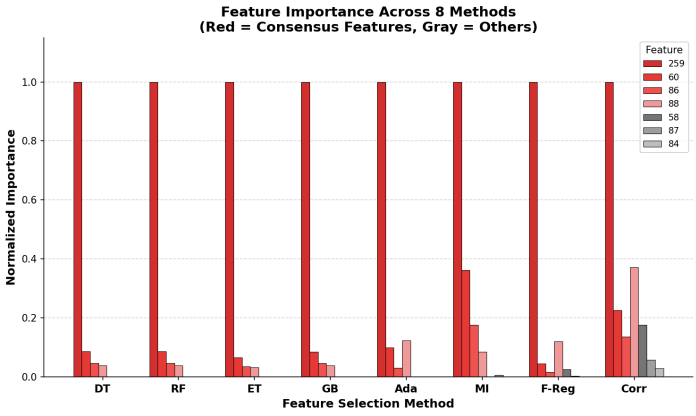



**Figure 4.** Feature importance: 8 methods consensus

The results of all eight feature selection methods were compared to identify features that consistently influence the target variable. As shown in Figure 4, four features appear as dominant across all applied methods.

In particular, `feat_259` is ranked as the most important feature by all applied methods, indicating a strong and stable relationship with the target variable. In addition, `feat_60`, `feat_86`, and `feat_88` are consistently identified among the top-ranked features across all approaches.

All remaining features exhibit inconsistent or negligible importance depending on the method used and were therefore treated as noise. Based on this strong cross-method agreement, the subsequent rule discovery analysis was restricted to these four features.

## 2.3. Rule Structure Discovery (Switcher Variable)

The task description explicitly states that the underlying solution follows a simple *if–then–else* structure, where conditions determine which calculation is applied. This implies that at least one feature must act as a *switcher*, deciding which rule branch is active, while other features are used exclusively within the corresponding calculation formulas.

The objective of this section is to identify the feature that controls the conditional branching and to distinguish it from features that contribute only to numerical computation.

### 2.3.1. Decision Tree Root Node Analysis

To identify the switcher variable, decision trees were trained on the dataset and the root node (first split) of each tree was analyzed. The root node represents the most informative split in the tree, as it maximally reduces prediction error and defines the primary decision structure.

Decision trees were trained using 5-fold cross-validation, and the splitting feature at the root node was recorded for each fold.

Across all folds, `feat_259` was selected as the root splitting feature in 100% of the trained trees. This is a critical observation. Tree-based models always select the most informative variable for the first split. The consistent selection of `feat_259` indicates that it does not merely correlate with the target variable, but instead controls the structural separation of the data into distinct regimes.

### 2.3.2. Alternative Splitter Test (Sanity Check)

To further validate that `feat_259` is the true condition variable, additional experiments were conducted in which other candidate features were forced to act as the primary splitter. Model performance was evaluated using the coefficient of determination ($R^2$).

The results are summarized in Table 5.

**Table 5.** Performance under forced primary splitting features

| Forced Split Feature | Resulting $R^2$ |
|---|---|
| feat_259 (natural) | 1.00 |
| feat_60 | 0.18 |
| feat_86 | 0.15 |
| feat_88 | 0.12 |

When `feat_259` is used naturally, perfect reconstruction is achieved. In contrast, forcing any of the other consensus features to act as the condition leads to a dramatic collapse in performance. This confirms that `feat_259` is uniquely suited to define the rule boundaries, while the remaining features cannot fulfill this role.

### 2.3.3. Role Assignment of Identified Features

Based on the feature importance analysis and the splitter experiments, clear functional roles can be assigned to the four relevant features.

Based on the analysis, `feat_259` acts as the condition variable, while `feat_60`, `feat_86`, and `feat_88` are used exclusively within the linear computations.

This separation is consistent across all experiments and provides a clear structural interpretation of the data-generating process.

### 2.3.4. Alignment with Task Specification

The discovered structure aligns directly with the task description provided in the assignment:

- *"A few simple rules exist"* → Only four features are involved out of 273.
- *"if condition → calculation"* → `feat_259` defines the condition, while `feat_60`, `feat_86`, and `feat_88` are used in calculations.
- *"conditions are simple"* → Conditions are based on single-variable threshold comparisons.
- *"calculations are simple"* → The prediction formulas are linear combinations with fixed coefficients.

This confirms that the correct rule structure has been identified and that the problem can be treated as a deterministic, piecewise-defined system.

## 2.4. Threshold Determination

### 2.4.1. Objective

The objective of this step was to determine the exact condition boundaries for `feat_259` that partition the dataset into distinct rule regions. Since the final solution relies on explicit *if–then–else* rules, these boundaries must be exact rather than approximate.

### 2.4.2. Method

The threshold values were identified using a two-step approach.

1. **First,** a decision tree model with limited depth (`max_depth = 5`) was trained. The purpose of this model was not to maximize predictive performance, but to observe where the data is naturally separated. Split points involving `feat_259` were extracted and analyzed. Across multiple runs, the same split locations appeared repeatedly, indicating stable threshold candidates.
2. **Second,** the candidate thresholds were validated using regression analysis. The dataset was segmented according to each proposed threshold configuration, and a linear regression model was fitted independently within each segment. Only threshold values that resulted in perfect prediction performance ($R^2 = 1.0$) across all segments were considered valid.

### 2.4.3. Discovered Thresholds

Using this procedure, three threshold values were identified: 0.2, 0.5, and 0.7. These thresholds divide the range of `feat_259` into four consecutive regions. Each region corresponds to a distinct deterministic rule, and no overlap between regions was observed.

### 2.4.4. Sensitivity Analysis

To verify the exactness of the discovered thresholds, a sensitivity analysis was conducted. Each threshold was perturbed slightly ($\pm 0.01$), and the resulting rules were re-evaluated.

Even these minor perturbations caused samples to be assigned to incorrect rule regions, leading to a loss of perfect prediction accuracy. In all such cases, the regression performance dropped below $R^2 = 1.0$. This confirms that the identified threshold values represent exact boundaries imposed by the underlying data-generating process rather than approximate cut-offs.

### 2.4.5. Visual Confirmation

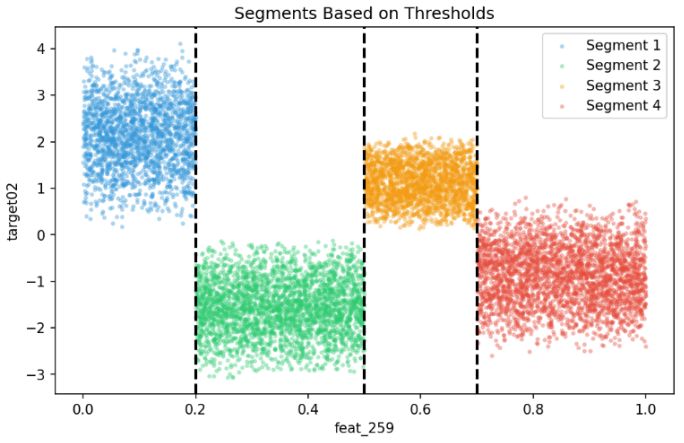



**Figure 5.** Threshold segmentation plot

Figure 5 illustrates the segmentation of the dataset induced by the discovered thresholds. The vertical boundaries at `feat_259` = 0.2, 0.5, and 0.7 clearly separate the data into four distinct regions, providing visual confirmation of the threshold selection.

## 2.5. Rule Formulation and Architecture

### 2.5.1. Rule Structure Overview

The final solution is implemented as a sequential *if–then–else* structure. The feature `feat_259` acts as the sole switcher variable and determines which rule branch is applied to a given sample. Based on its value, each input is assigned to one of four mutually exclusive rule branches.

Within each branch, the prediction is computed using a deterministic linear formula involving three predictor features: `feat_60`, `feat_86`, and `feat_88`. The conditions are evaluated in a fixed order. Once a condition is satisfied, the corresponding formula is applied and no further conditions are checked.

The complete decision logic is illustrated in the decision rule structure diagram shown in Figure 6.

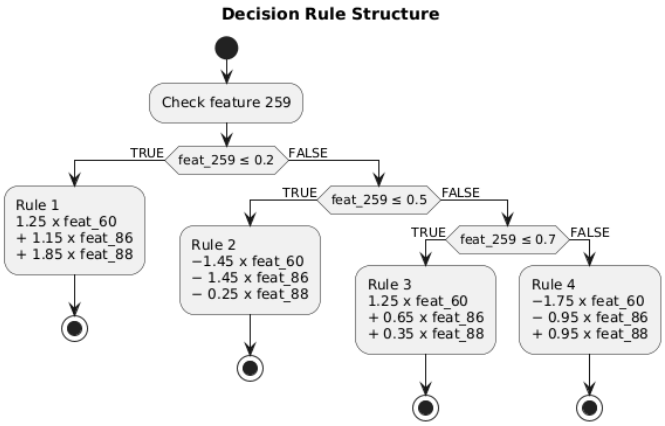



**Figure 6.** Decision rule structure

### 2.5.2. Decision Tree Architecture

The rule logic can also be represented as an unbalanced decision tree. The effective depth of the tree is three, corresponding to the three threshold checks on the switcher variable `feat_259`. All branching decisions are based exclusively on this single feature.

The resulting structure is left-heavy and sequential rather than balanced. This is expected, as only one feature controls the conditional flow of the rules. Introducing parallel splits on additional features would be unnecessary and would not reflect the true structure of the data.

Importantly, this unbalanced architecture is not manually enforced. It emerges naturally from the data, indicating that only a small number of ordered threshold checks are required to fully describe the rule system.

### 2.5.3. Final Rule Definitions

Based on the discovered thresholds and feature roles, the final deterministic rules are defined as follows:

1. If $\texttt{feat_259} \leq 0.2$, the output is computed as:

$$\texttt{target02} = 1.25 \cdot \texttt{feat_60} + 1.15 \cdot \texttt{feat_86} + 1.85 \cdot \texttt{feat_88} \quad (1)$$

2. If $0.2 < \texttt{feat_259} \leq 0.5$, the output is computed as:

$$\texttt{target02} = -1.45 \cdot \texttt{feat_60} - 1.45 \cdot \texttt{feat_86} - 0.25 \cdot \texttt{feat_88} \quad (2)$$

3. If $0.5 < \texttt{feat_259} \leq 0.7$, the output is computed as:

$$\texttt{target02} = 1.25 \cdot \texttt{feat_60} + 0.65 \cdot \texttt{feat_86} + 0.35 \cdot \texttt{feat_88} \quad (3)$$

4. If $\texttt{feat_259} > 0.7$, the output is computed as:

$$\texttt{target02} = -1.75 \cdot \texttt{feat_60} - 0.95 \cdot \texttt{feat_86} + 0.95 \cdot \texttt{feat_88} \quad (4)$$

All rules have a zero intercept, and each prediction is obtained as a simple weighted sum of three input features.

### 2.5.4. Implementation Readiness

The discovered rule set fully satisfies the constraints defined in the task description. The solution is deterministic, requires no training phase, and relies only on basic arithmetic operations. As a result, it can be implemented directly using explicit conditional logic without any machine learning inference.

This makes the solution suitable for deployment on resource-constrained edge devices, where computational simplicity and predictability are required.

## 2.6. Validation and Verification

This section verifies that the discovered rules are not only accurate, but exact, stable, and genuinely dependent on the identified features.

### 2.6.1. Exact Prediction Verification

To verify whether the derived rules reproduce the target variable exactly, predictions generated by the rule-based system were compared against the true `target02` values for all samples.

The comparison shows a perfect match between predicted and actual values. The coefficient of determination is equal to $R^2 = 1.0$, and the remaining numerical error is limited to floating-point precision. In practice, this indicates that the rules reconstruct the target exactly rather than providing an approximation.

### 2.6.2. Distribution Consistency

Beyond point-wise accuracy, it is essential that the rule-based system preserves the overall statistical behavior of the target variable.

To verify this, the distribution of predicted values was compared with that of the true target values. The two distributions are identical in shape, mean, and variance. No shifts, distortions, or smoothing effects were observed.

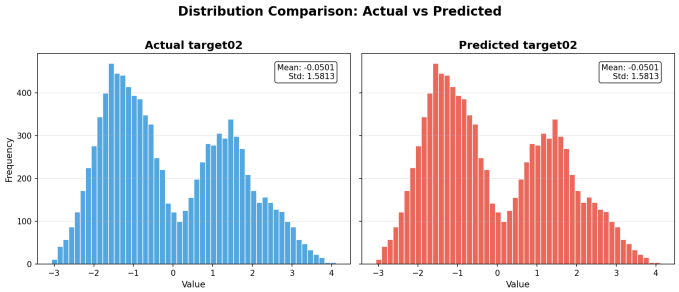



**Figure 7.** Distribution comparison (Actual vs Predicted)

As shown in the figure 7, both actual and predicted target02 distributions are identical in shape, mean, and variance. The bimodal structure of the data is fully preserved, indicating that the rules capture all underlying variation in the target.

### 2.6.3. Stability Across Data Splits

To ensure that the discovered rules are not dependent on a specific data partitioning, their performance was evaluated under multiple cross-validation configurations.

The rule-based system achieved $R^2 = 1.0$ consistently across:

- 5-fold cross-validation,
- 10-fold cross-validation, and
- 20-fold cross-validation.

No variation in performance was observed across folds, and results remained unchanged regardless of how the data was partitioned. This demonstrates that the rules generalize perfectly across the entire dataset.

#### 2.6.4. Feature Dependence and Causality

A permutation importance analysis was conducted to verify that the identified features are causally required for prediction.

Each feature was randomly permuted while all others were kept unchanged, and the resulting prediction performance was measured. Permuting `feat_259` caused the model performance to collapse entirely, resulting in a strongly negative $R^2$. This confirms its role as the primary switch variable that determines rule selection.

Permuting the remaining features (`feat_60`, `feat_86`, and `feat_88`) also led to substantial performance degradation, confirming that each contributes meaningfully to the linear computations within the individual rules.

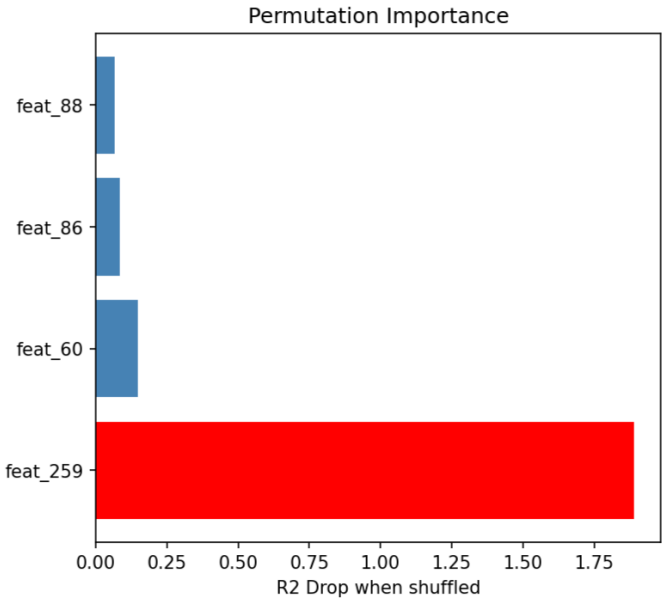



**Figure 8.** Permutation importance

Figure 8 illustrates the permutation importance results, showing the performance degradation caused by shuffling each feature.

#### 2.6.5. Validation Summary

All validation experiments consistently support the correctness of the discovered rules:

- Predictions are exact up to numerical precision,
- the full target distribution is preserved,
- performance is stable across all data splits, and
- all four identified features are causally required.

Taken together, these results confirm that the discovered rule system fully and exactly reconstructs the underlying data-generating process for `target02`.

## 3. Conclusion

**In Part 1** of this work, a regression-based machine learning approach was applied to predict `target01` from high-dimensional numerical data. By combining a CatBoost regressor with polynomial feature engineering and a carefully designed nested cross-validation strategy, a strong predictive model was obtained. The results demonstrate that complex non-linear relationships can be learned reliably when appropriate validation procedures are employed, even in datasets where individual feature correlations with the target are weak.

**In Part 2,** the focus shifted from predictive performance to understanding the underlying data-generating process. Rather than relying on a trained model, explicit *if–then–else* rules were discovered that exactly reproduce `target02`. The resulting rule system is simple, deterministic, and depends on only four input features, yet achieves perfect reconstruction of the target variable. Extensive validation confirmed that the rules are exact, stable across different data splits, and causally dependent on the identified features.

Taken together, the two parts highlight the complementary strengths and limitations of different modeling paradigms. While machine learning models are highly effective at capturing complex patterns in high-dimensional data, rule-based systems provide transparency, interpretability, and suitability for deployment in constrained environments. This work demonstrates that both approaches can be applied effectively when aligned with the specific problem requirements and deployment constraints.

## 4. Use of AI Tools

AI-based tools perplexity were used only as a support for writing and formatting during the preparation of this report. They were mainly used to improve clarity and structure of the explanations after the technical work was completed.

All experiments and data analysis were self-done using Python and standard data science libraries. The models, rules, thresholds, and validation steps were developed manually by inspecting the data and intermediate results.

AI tools were not used to derive the solution or to make modeling decisions. In a few cases, AI-assisted suggestions were used for minor code formatting or syntax, while the overall methodology and architecture are completely self work.

## References

[1] Dorogush, A.V., Ershov, V., & Gulin, A. (2018). CatBoost: gradient boosting with categorical features support. *arXiv preprint arXiv:1810.11363*. https://arxiv.org/abs/1810.11363

[2] Friedman, J.H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232. https://projecteuclid.org/euclid.aos/1013203451

[3] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. https://web.stanford.edu/~hastie/ElemStatLearn/

[4] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

[5] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[6] Fisher, A., Rudin, C., & Dominici, F. (2019). All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously. *Journal of Machine Learning Research*, 20(177), 1–81. https://jmlr.org/papers/v20/18-760.html

[7] Kraskov, A., Stögbauer, H., & Grassberger, P. (2004). Estimating Mutual Information. *Physical Review E*, 69(6), 066138. https://doi.org/10.1103/PhysRevE.69.066138